

Trabalho Prático: Sudoku Genérico como CSP

Contexto

O Sudoku clássico — uma grelha $n^2 \times n^2$ onde cada linha, cada coluna e cada bloco $n \times n$ tem de conter todos os valores de 1 a n^2 sem repetições — é um exemplo canónico de **problema de satisfação de restrições (CSP)**: a "regra" é sempre a mesma (um conjunto de células tem de ter valores todos diferentes), o que muda de linha para linha, de coluna para coluna e de bloco para bloco é apenas **que células pertencem a esse conjunto**.

Isso sugere uma abstração única — um grupo de células com a restrição "todos diferentes", opcionalmente com algumas células já fixas a um valor — a partir da qual linhas, colunas, blocos e ainda outras variantes de Sudoku (diagonais, regiões irregulares, grelhas sobrepostas, etc.) podem ser todas construídas sem duplicar lógica de restrição nenhuma.

Este é um problema de **modelação e resolução de CSP**. Cabe-te a ti escolher a técnica de resolução e justificá-la — o enunciado não fornece código de modelação nem de apresentação de resultados, apenas a interface que o teu notebook tem de expor (secção seguinte) para poder ser testado automaticamente.

Objetivo

Construir, num notebook Marimo, um gerador/resolvedor de Sudoku $n^2 \times n^2$ (com n parametrizável, tipicamente $n = 3$) que:

1. representa qualquer **grupo de células com restrição "todos diferentes"** através de uma classe genérica (secção " `box` — grupo genérico de células"),
2. constrói **linhas, colunas e blocos** como casos particulares dessa classe genérica — os blocos através de uma especialização dedicada a blocos $n \times n$, as linhas e colunas através de uma especialização dedicada a sequências retas de células (secção " `cube` e `path` "),
3. gera **aleatoriamente** um subconjunto de células já preenchidas (as "pistas" iniciais do puzzle), usando a mesma abstração genérica (secção "Geração aleatória de pistas"),
4. monta o modelo completo (linhas + colunas + blocos + pistas) e o resolve como CSP, devolvendo a grelha preenchida ou sinalizando que não há solução (secção "Resolução").

Requisitos obrigatórios

O teu notebook tem de expor, com este comportamento, os seguintes elementos (os nomes propostos abaixo são sugestões que facilitam a correção automática — podes usar outros, desde que documentes a correspondência):

`box` — grupo genérico de células (R1)

Uma classe que representa **qualquer** conjunto de células da grelha às quais se aplica a restrição "todos os valores diferentes", com algumas delas possivelmente já fixas:

- guarda internamente uma associação `(linha, coluna) → valor` ou `None` (`None` = célula livre; um inteiro = célula fixa/pinada a esse valor);
- um construtor que aceita opcionalmente esse conjunto inicial de células (vazio por omissão);
- um método `add(i, j, val=None)` que acrescenta a célula `(i, j)` ao grupo, opcionalmente fixando-a a `val`, e que **rejeita** (levanta exceção) coordenadas fora da grelha ou valores fora do intervalo $[1, n^2]$;
- uma forma de obter a representação do grupo como matriz $n^2 \times n^2$, com zeros nas células não pertencentes ao grupo ou não fixas, e o valor fixo nas restantes.

Esta classe **não deve saber nada** sobre linhas, colunas, blocos ou Sudoku — só sabe lidar com "um conjunto de células, algumas fixas". Essa generalidade é o que te vai permitir, mais tarde, tratar da mesma forma linhas, colunas, blocos, pistas aleatórias e (nas extensões opcionais) diagonais ou regiões irregulares.

`cube` e `path` — duas formas concretas de grupo (R2, R3)

A partir da classe genérica, define duas especializações:

- **R2.** Um grupo que representa o **bloco** $n \times n$ cujo canto superior esquerdo é a célula $(i \cdot n, j \cdot n)$, parametrizado pelos índices de bloco (i, j) com $0 \leq i, j < n$.
- **R3.** Um grupo que representa o **troço reto** (horizontal ou vertical) de células entre duas coordenadas `inicio` e `fim`, inclusive — tem de funcionar tanto para `fim` "depois" de `inicio` como "antes" (ou seja, percorrer a sequência em qualquer sentido).

Geração aleatória de pistas (R4)

Uma função que devolve um grupo (`box`) com k células escolhidas aleatoriamente na grelha, cada uma fixa a um valor também escolhido aleatoriamente em $[1, n^2]$ (k deve ter um valor por omissão razoável, por exemplo da ordem de n). Repara que esta

função **não precisa de nenhuma classe nova** — o resultado é, de novo, apenas um `box`.

Modelo e resolução (R5, R6)

- **R5.** Um modelo de CSP para a grelha $n^2 \times n^2$, com uma variável inteira por célula, cada uma no intervalo $[1, n^2]$; um método que recebe **um número arbitrário de grupos** (`box`, `cube`, `path`, ou pistas aleatórias — o modelo não deve distinguir a sua origem) e, para cada um, impõe que as suas células sejam todas diferentes e fixa as que tiverem valor atribuído; e um método de resolução que devolve a grelha preenchida ou sinaliza, de forma distinguível, que o puzzle não tem solução.
- **R6.** Um Sudoku $n^2 \times n^2$ completo é montado juntando: todas as linhas, todas as colunas, todos os blocos $n \times n$ e (pelo menos) um grupo de pistas aleatórias — e resolvido.

Como testar/validar

O teu notebook (ou um ficheiro de testes à parte) tem de verificar automaticamente, para uma grelha resolvida:

- que cada linha, cada coluna e cada bloco $n \times n$ contém exatamente os valores $1 \dots n^2$, sem repetições;
- que as células fixadas pelas pistas aleatórias mantêm, na solução, o valor com que foram fixadas;
- que `add` (ou equivalente) rejeita coordenadas fora da grelha e valores fora de $[1, n^2]$.

Corre o fluxo completo (gerar pistas aleatórias → montar linhas + colunas + blocos + pistas → resolver → validar) pelo menos uma vez com $n = 3$ (Sudoku clássico 9×9) e confirma que também funciona com outro valor de n (ex.: $n = 2$, grelha 4×4), para garantir que nada está fixo a 9×9 no teu código.

O que é deixado ao teu critério

O enunciado define **que abstrações** o notebook tem de expor e **que comportamento** têm de ter, não **como** as deves implementar. Ficam ao teu critério, desde que justificadas no notebook:

- a técnica e biblioteca de resolução do CSP (CP-SAT do OR-Tools é a sugestão da disciplina, mas és livre de escolher outra abordagem de Lógica Computacional, justificando a escolha);
- a estrutura de dados interna do grupo genérico (dicionário, matriz esparsa, etc.);

- a forma de apresentar a grelha resultante (texto, tabela, `mo.ui`, gráfico — o que achares mais claro);
- o comportamento exato quando o puzzle gerado aleatoriamente não tem solução (podes, por exemplo, tentar novas pistas aleatórias até obteres um puzzle solúvel, ou simplesmente reportar o insucesso — justifica a escolha).

Extensões opcionais (bónus)

A generalidade do `box` é o que torna estas extensões possíveis sem tocar no modelo CSP em si — cada uma acrescenta apenas **novos grupos** de células:

- **Sudoku diagonal (X-Sudoku)**: acrescenta um grupo (`box`, sem precisar de nova subclasse) para cada uma das duas diagonais principais, também elas restritas a "todos diferentes".
- **Sudoku irregular (jigsaw)**: substitui os blocos $n \times n$ regulares por regiões de forma arbitrária mas do mesmo tamanho, cada uma representada como um `box` construído célula a célula em vez de por `cube`.
- **Hyper-Sudoku / Windoku**: acrescenta 4 blocos extra (também `box`, de forma semelhante a `cube` mas sem estarem alinhados com a grelha $n \times n$ de blocos) sobrepostos aos existentes.
- **Escala**: mostra que o teu código funciona (talvez mais devagar) para $n = 6$ (grelha 36×36) sem alterações, e discute os limites de desempenho que encontres.
- **Sudoku tridimensional** define a estrutura de "boxes" numa grelha $n^2 \times n^2 \times n^2$.

```
In [ ]: import marimo as mo
import random
from ortools.sat.python import cp_model
```

```
In [ ]: #R1
class Box:
    # 0 def precisa ter esse recuo para fazer parte da classe Box
    def __init__(self, n, celulas=None):
        self.n = n
        self.tamanho = n * n

        self.celulas = {}
        if celulas:
            for (i,j), v in celulas.items():
                self.add(i,j,v)
    def add(self, i, j, val=None):
        if not(0 <= i < self.tamanho and 0 <= j < self.tamanho):
            raise ValueError(f"CORDENACAO ({i},{j}) fora da grelha
{self.tamanho} * {self.tamanho}")
        if val is not None and not(1 <= val <= self.tamanho):
            raise ValueError(f"Valor {val} fora do intervalo[1,
{self.tamanho}]")
        self.celulas[(i,j)] = val
```

```

def matriz(self):
    grelha = [[0] * self.tamanho for _ in range(self.tamanho)]
    for (i, j), v in self.celulas.items():
        if v is not None:
            grelha[i][j] = v
    return grelha

#R2

#identificar qual matriz interna (bi,bj) da matriz total
class Cube(Box):
    def __init__(self, n, bi, bj):
        super().__init__(n)
        if not (0 <= bi < n and 0 <= bj < n):
            raise ValueError(f"Indices de bloco ({bi},{bj}) invalido para
n={n}")

        base_i = bi*n
        base_j = bj*n

        for di in range(n):
            for dj in range(n):
                self.add(base_i + di, base_j + dj)

#R3
class Path(Box):
    def __init__(self, n, inicio, fim):
        super().__init__(n) # chama o construtor do Box
        (i1, j1), (i2, j2) = inicio, fim

        if i1 != i2 and j1 != j2: # não está na mesma linha
nem coluna?
            raise ValueError(f"{inicio} e {fim} não formam uma linha reta")

        if i1 == i2: # CAMINHO HORIZONTAL (mesma
linha)
            passo = 1 if j2 >= j1 else -1 # anda para a frente ou para
trás
            for j in range(j1, j2 + passo, passo):
                self.add(i1, j)
        else: # CAMINHO VERTICAL (mesma
coluna)
            passo = 1 if i2 >= i1 else -1
            for i in range(i1, i2 + passo, passo):
                self.add(i, j1)

#R4
def pistas_aleatorias(n, k=None):
    """Devolve um Box com k células aleatórias fixas."""
    if k is None:
        k = n
    if k > n**4:
        raise ValueError(f"k={k} excede o número de células da grelha
({n**4})")
    box = Box(n)

```

```

    escolhidas = set()
    while len(escolhidas) < k:
        i = random.randint(0, n * n - 1)
        j = random.randint(0, n * n - 1)
        if (i, j) not in escolhidas:
            escolhidas.add((i, j))
            box.add(i, j, random.randint(1, n * n))
    return box

#R5

class SudokuCSP:

    def __init__(self, n):
        self.n = n
        self.tamanho = n*n
        self.model = cp_model.CpModel()

        self.vars = {}
        for i in range(self.tamanho):
            for j in range(self.tamanho):
                self.vars[(i, j)] = self.model.NewIntVar(
1, self.tamanho, f"celula_{i}_{j}")

    def adicionar_grupos(self, *grupos):
        """Recebe um número arbitrário de grupos e impõe as suas
restrições."""
        for grupo in grupos:
            # "todos diferentes": uma variável por célula do grupo
            variaveis = [self.vars[(i, j)] for (i, j) in grupo.celulas]
            self.model.AddAllDifferent(variaveis)
            # fixar as células que têm valor atribuído (as pistas)
            for (i, j), v in grupo.celulas.items():
                if v is not None:
                    self.model.Add(self.vars[(i, j)] == v)

    def resolver(self):
        """Devolve a grelha resolvida (lista de listas) ou None se
insolúvel."""
        solver = cp_model.CpSolver()
        status = solver.Solve(self.model)
        if status in (cp_model.OPTIMAL, cp_model.FEASIBLE):
            return [
                [solver.Value(self.vars[(i, j)]) for j in
range(self.tamanho)]
                for i in range(self.tamanho)
            ]
            return None

#R6

def sudoku_completo(n, k=None, seed=None):
    """Gera pistas, monta linhas+colunas+blocos e resolve. Devolve grelha
ou None."""
    if seed is not None:
        random.seed(seed)

```

```

modelo = SudokuCSP(n)

# todos os blocos n×n
for bi in range(n):
    for bj in range(n):
        modelo.adicionar_grupos(Cube(n, bi, bj))

# todas as linhas e todas as colunas (via Path)
for idx in range(n * n):
    modelo.adicionar_grupos(
        Path(n, (idx, 0), (idx, n * n - 1)), # linha idx
        Path(n, (0, idx), (n * n - 1, idx)), # coluna idx
    )

# pistas aleatórias
pistas = pistas_aleatorias(n, k)
modelo.adicionar_grupos(pistas)

return modelo.resolver(), pistas

```

```

In [ ]: def validar_solucao(grelha, pistas, n):
    assert grelha is not None, "sem solução para validar"
    t = n * n
    esperado = set(range(1, t + 1))

    # a) cada linha, coluna e bloco tem exatamente 1..n²
    for i in range(t):
        assert set(grelha[i]) == esperado, f"Linha {i} inválida"
        assert {grelha[r][i] for r in range(t)} == esperado, f"Coluna {i} inválida"
        for bi in range(n):
            for bj in range(n):
                bloco = {grelha[bi*n + di][bj*n + dj]
                           for di in range(n) for dj in range(n)}
                assert bloco == esperado, f"Bloco ({bi},{bj}) inválido"

    # b) as pistas mantêm os valores fixados
    for (i, j), v in pistas.celulas.items():
        if v is not None:
            assert grelha[i][j] == v, f"Pista ({i},{j})={v} não respeitada"

    return True

```

```

In [ ]: # Tenta criar um sudoku aleatório até dar certo
solucao_aleatoria = None
solucao_aleatorial=None

print("Procurando um tabuleiro aleatório válido...")
tentativas1 = 0
tentativas2= 0

while solucao_aleatoria is None and tentativas1 <20:
    tentativas1 += 1

```

```

    # Chamamos sem o seed! Totalmente aleatório.
    solucao_aleatoria, pistas1 = sudoku_completo(n=3, k=3)

print(f" Sucesso! Jogo válido encontrado após {tentativas1} tentativas.")

while solucao_aleatoria is None and tentativas2 <20:
    tentativas2 +=1
    solucao_aleatoria1, pistas2= sudoku_completo(n=2,k=2)

print(f" Sucesso! Jogo válido encontrado após {tentativas2} tentativas.")

# Agora sim, temos garantia que a solução existe e podemos imprimir
for linha in solucao_aleatoria:
    print(linha)

print("\n")

for linha1 in solucao_aleatoria1:
    print(linha1)

```

```

In [ ]: def testar_fluxo(n, k=None, tentativas=100):
        for seed in range(tentativas):
            grelha, pistas = sudoku_completo(n, k, seed=seed)
            if grelha is not None:
                validar_solucao(grelha, pistas, n)
                return seed, grelha
            raise AssertionError(f"Nenhuma solução em {tentativas} sementes (n={n})")

        for n in (3, 2):
            _seed, _g = testar_fluxo(n)
            print(f"n={n}: {len(_g)} linhas x {len(_g[0])} colunas, max={max(max(l)
            for l in _g)}")
            for _linha in _g:
                print(*_linha)

```

```

In [ ]: # Coordenada fora da grelha
try:
    Box(3).add(10, 0)
    print("❌ add aceitou coordenada inválida – BUG!")
except ValueError:
    print("✅ add rejeitou coordenada fora da grelha")

# Valor fora de [1, n²]
try:
    Box(3).add(0, 0, 15)
    print("❌ add aceitou valor inválido – BUG!")
except ValueError:
    print("✅ add rejeitou valor fora de [1, 9]")

# Caso válido continua a funcionar
Box(3).add(4, 4, 7)
print("✅ add aceita coordenada e valor válidos")

```

```
In [ ]: # Duas pistas iguais na MESMA linha → impossível
modelo = SudokuCSP(2)
modelo.adicionar_grupos(Path(2, (0, 0), (0, 3)))
modelo.adicionar_grupos(pistas_aleatorias(2, k=0)) # sem pistas
aleatórias

# fixa manualmente duas células da linha 0 ao mesmo valor 1
grupo_conflito = Box(2)
grupo_conflito.add(0, 0, 1)
grupo_conflito.add(0, 1, 1)
modelo.adicionar_grupos(grupo_conflito)

print("Resultado:", modelo.resolver()) # deve ser None
assert modelo.resolver() is None
print("✅ Puzzle insolúvel devolveu None (sinal distinguível)")
```

```
In [ ]: # Teste de Puzzle Impossível (duas pistas iguais na mesma linha)
pistas_impossiveis = Box(3, {(0, 0): 5, (0, 1): 5}) # 0 '5' repete na
linha 0

modelo_falho = SudokuCSP(3)
modelo_falho.adicionar_grupos(
    Path(3, (0,0), (0,8)), # Restrição da linha 0
    pistas_impossiveis      # Inserção das pistas falhas
)

resultado = modelo_falho.resolver()
print("Tentativa de resolver puzzle com erro de regras:")
print("Resultado:", resultado) # A saída esperada é None
```

Justificação das Opções de Implementação

Técnica de resolução: Optámos pelo CP-SAT do OR-Tools, a sugestão da disciplina. Trata-se de um solver de programação por restrições que modela nativamente variáveis inteiras com domínios finitos e restrições `AllDifferent`, sendo muito eficiente para este tipo de CSP.

Estrutura de dados (Box): Usámos um dicionário `(i, j) → valor` ou `None`, em vez de uma matriz densa:

1. Os grupos são esparsos na grelha global (uma linha ocupa n^2 de n^4 células; as pistas, ainda menos).
2. Permite representar grupos de forma arbitrária (essencial para extensões como diagonais ou jigsaw).
3. A verificação de pertença é $O(1)$ e a iteração é $O(k)$, com k o número de células do grupo.

Apresentação dos resultados: A grelha é impressa como linhas de inteiros em texto, por ser a forma mais direta de verificar a correção; `Box.matriz()` complementa a visualização de grupos individuais.

Tratamento de insucesso: o solver reporta o insucesso devolvendo None, sem regenerar pistas automaticamente — assim, eventuais erros de modelação não ficam mascarados. Nos testes, varremos um conjunto de sementes (até 100) até encontrar um puzzle solúvel, garantindo reprodutibilidade através da seed; caso nenhuma seed produza solução, o teste falha explicitamente

Utilização de ferramentas LLM

Este trabalho foi desenvolvido com apoio do assistente **Claude**, como exigido no ponto 6.2 das normas gerais dos trabalhos práticos.

O diálogo completo encontra-se no <https://claude.ai/chat/a193f5be-fd7c-4c4c-9008-8f026eb10122> deste repositório. A LLM foi usada para:

- explicação de conceitos de Python (classes, herança, `self`, `super()`, parâmetros opcionais, dicionários);
- fornecimento de esqueletos de código para cada requisito (R1–R6);
- ajuda na depuração de erros (typo `seft`, tuplas acidentais, lógica da validação do `add`).

****Trabalho realizado Por:** Wu Hou Pan Fábio Miguel Costa Reis .

In []: